

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Алгоритмы и структуры данных»
ТЕМА: РЕАЛИЗАЦИЯ И ИССЛЕДОВАНИЕ СТРУКТУРЫ ДАННЫХ RORE

Студент гр. 4388

Гриценко К.В.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

Цель работы.

Реализация структуры данных **Rope**, с различными методами, позволяющими соединять, разбивать, вставлять, удалять данные в структуре, а также получать данные по индексу.

Задание.

Реализация

Вам необходимо реализовать структуру данных **Rope** (верёвка). Она должна иметь следующие методы:

1. **concat** — конкатенация с другой верёвкой;
2. **split** — разбиение верёвки на две по индексу;
3. **index** — получение символа по индексу;
4. **insert** — вставка строки в исходную строку, начиная с заданной позиции.
5. **delete** — удаление подстроки заданной длины из исходной строки, начиная с заданной позиции.

Исследование

После успешного решения задачи в рамках курса проведите исследование данной структуры на различных размерах данных (10/1000/100000) и размерах листовых узлов, сравнив полученные результаты с теоретической оценкой (для лучшего, среднего и худшего случаев).

Примечание:

При решении задачи в курсе используйте максимальный размер листового узла, равный восьми. Обратите внимание на пример.

Формат ввода

Первая строка содержит текст, состоящий из символов ASCII. На следующей строке дан индекс и через пробел — текст, который необходимо вставить по этому индексу. На третьей строке через пробел даны индекс начала и длина подстроки, которую нужно удалить из текста.

Формат вывода

Выводятся листья слева направо в формате “Leaf i: <строка, хранящаяся в листе>”, затем нелистовые узлы в порядке обхода в глубину в формате “Node i: <вес узла>”, затем выводится текст, хранящийся в верёвке. Такой вывод ожидается трижды: после построения верёвки, после вставки и после удаления.

Функции:

__init__(self, string_data=None)

- **Назначение:** Конструктор узла дерева
- **Инициализирует:** левого и правого потомков, строковые данные, вес и высоту узла
- **Логика:** Для листовых узлов вес равен длине строки, для внутренних -0

is_leaf(self)

- **Назначение:** Проверка типа узла
- **Возвращает:** *True* если узел содержит строковые данные (является листом)
- **Критерий:** *string_data is not None*

update_height(self)

- **Назначение:** Обновление высоты узла на основе высот потомков
- **Формула:** $\max(\text{left_child.height}, \text{right_child.height}) + 1$
- **Использование:** После изменений структуры дерева

balance_factor(self)

- **Назначение:** Вычисление коэффициента балансировки узла
- **Формула:** $\text{left_child.height} - \text{right_child.height}$
- **Назначение:** Определение необходимости балансировки

__init__(self, initial_string="")

- **Назначение:** Конструктор объекта *Rope*

- **Действие:** Создает корневой узел путем построения дерева из начальной строки
- **Вызывает:** `_build_tree(initial_string)`
`_build_tree(self, s)`
- **Назначение:** Рекурсивное построение сбалансированного дерева из строки
- **Алгоритм:**
 - Если длина строки $\leq LEAF_SIZE$ (8) - создает листовой узел
 - Иначе делит строку пополам и рекурсивно строит левое и правое поддеревья
- **Балансировка:** Вызывает `_balance_node` для каждого внутреннего узла
`get_char_at(self, index)`
- **Назначение:** Получение символа по заданному индексу
- **Алгоритм:**
 - Спуск по дереву с учетом весов узлов
 - Если индекс < веса узла - переход в левое поддерево
 - Иначе вычитание веса и переход в правое поддерево
- **Сложность:** $O(\log n)$
`_compute_length(self, node)`
- **Назначение:** Рекурсивное вычисление длины строки в поддереве
- **Для листа:** Возвращает `len(node.string_data)`
- **Для внутреннего узла:** Сумма длин левого и правого поддеревьев
- **Использование:** При обновлении весов узлов
`_rotate_right(self, y_node)`
- **Назначение:** Правый поворот вокруг узла `y`
- **Операции:**
 - Сохранение левого потомка (`x`) и его правого поддерева (`T2`)
 - Перемещение `x` на место `y`
 - Присоединение `y` как правого потомка `x`
 - Присоединение `T2` как левого потомка `y`

- **Обновление:** Весов и высот затронутых узлов

_rotate_left(self, x_node)

- **Назначение:** Левый поворот вокруг узла x
- **Операции:**
 - Сохранение правого потомка (y) и его левого поддерева ($T2$)
 - Перемещение y на место x
 - Присоединение x как левого потомка y
 - Присоединение $T2$ как правого потомка x

- **Обновление:** Весов и высот затронутых узлов

_balance_node(self, node)

- **Назначение:** Балансировка узла согласно правилам AVL-дерева
- **Обрабатываемые случаи:**
 - Left-Left: правый поворот
 - Right-Right: левый поворот
 - Left-Right: левый поворот левого потомка + правый поворот
 - Right-Left: правый поворот правого потомка + левый поворот

_split_tree(self, node, index)

- **Назначение:** Рекурсивное разделение дерева на две части по индексу
- **Для листа:** Физическое разделение строки на две части
- **Для внутреннего узла:**
 - Если индекс $\leq$ веса - разделение левого поддерева
 - Иначе разделение правого поддерева с корректировкой индекса
- **Возвращает:** Кортеж (левое_дерево, правое_дерево)

split_at(self, index)

- **Назначение:** Публичный интерфейс для разделения веревки
- **Действие:** Вызывает *_split_tree* и создает два новых объекта **Rope**
- **Возвращает:** Кортеж (*left_rope*, *right_rope*)

concatenate(self, other_rope)

- **Назначение:** Объединение текущей веревки с другой
- **Алгоритм:**

- Создание нового внутреннего узла
- Назначение текущего корня левым потомком
- Назначение корня другой веревки правым потомком
- Вычисление веса и балансировка

insert_at(self, index, string_to_insert)

- **Назначение:** Вставка строки в заданную позицию
- **Алгоритм:**
 1. Разделение текущей веревки в позиции *index*
 2. Создание новой веревки из вставляемой строки
 3. Последовательная конкатенация: *left + middle + right*

delete_range(self, start_index, length_to_delete)

- **Назначение:** Удаление диапазона символов
- **Алгоритм:**
 1. Разделение в *start_index* на *left* и *temp*
 2. Разделение *temp* в *length_to_delete* на *middle* и *right*
 3. Удаление *middle*
 4. Конкатенация *left* и *right*

print_structure(self)

- **Назначение:** Вывод структуры веревки для отладки
- **Отображает:** Строки листьев, веса внутренних узлов, полное содержимое

traverse_subtree(node)

- **Назначение:** Рекурсивный обход дерева для сбора данных
- **Собирает:**
 - *leaf_strings* - строки всех листьев
 - *internal_weights* - веса внутренних узлов
 - Полное содержимое веревки

main()

- **Назначение:** Логика работы программы
- **Логика:**

1. Чтение входных данных (исходная строка, параметры вставки и удаления)
2. Создание объекта **Rope**
3. Последовательное выполнение операций: вывод - вставка - вывод
- удаление - вывод
4. Демонстрация работы всех основных функций

Структурные особенности реализации:

- **LEAF_SIZE = 8** - максимальный размер строки в листовом узле
- **AVL-балансировка** - гарантия сбалансированности дерева
- **Рекурсивные алгоритмы** - для всех операций обхода и модификации

Принципы работы:

1. **Разделение при превышении** - строки длиной $> \text{LEAF_SIZE}$ разделяются рекурсивно
2. **Балансировка после операций** - каждая модификация дерева сопровождается проверкой баланса
3. **Вес узлов** - для эффективной навигации при операциях доступа по индексу
4. **Ленивые вычисления** - веса пересчитываются только при необходимости

Эффективность:

- **Вставка/удаление:** $O(\log n)$
- **Доступ по индексу:** $O(\log n)$
- **Конкатенация:** $O(1)$ (амортизировано)
- **Память:** $O(n)$ (для хранения всех узлов)

Тестирование программы приведено в приложении Б.

Исследование.

Структура данных **Rope** представляет собой сбалансированное бинарное дерево, предназначенное для эффективной работы с большими строками. Его основная идея заключается в том, чтобы разбивать строку на фрагменты (листья) и организовывать их в дерево, где каждый внутренний

узел хранит суммарную длину всех символов в своём левом поддереве (вес). Это позволяет выполнять операции вставки, удаления, разделения и конкатенации без необходимости копирования всей строки. В реализации используется AVL-балансировка, что гарантирует высоту дерева $O(\log(n))$ и, как следствие, логарифмическую сложность ключевых операций.

Конструктор `__init__` и методы `is_leaf`, `update_height` и `balance_factor` выполняются за $O(1)$, так как они лишь инициализируют поля, проверяют состояние или вычисляют простые арифметические выражения на основе высот потомков.

Построение всей структуры **Rope** из строки происходит в методе `_build_tree`. Он рекурсивно делит входную строку пополам, пока её длина не станет меньше или равна `LEAF_SIZE` (в данном случае 8). Для каждой части создаётся листовой узел. Поскольку каждый символ исходной строки попадает ровно в один лист, а количество создаваемых внутренних узлов пропорционально количеству листьев, общая сложность построения составляет $O(n)$, где n - длина строки. Метод `_compute_length`, который используется для расчёта веса узла, также имеет линейную сложность $O(k)$ относительно числа узлов в поддереве, так как он рекурсивно обходит всё поддерево.

Одной из ключевых операций является доступ к символу по индексу - метод `get_char_at`. Алгоритм спускается от корня к листу, используя вес узлов для навигации: если искомый индекс меньше веса текущего узла, поиск продолжается в левом поддереве, иначе индекс уменьшается на вес и поиск идёт в правом поддереве. Поскольку дерево сбалансировано, его высота равна $O(\log(n))$, и на каждом уровне выполняется константное количество операций. Таким образом, сложность доступа по индексу составляет $O(\log(n))$.

Для поддержания сбалансированности дерева используются методы поворотов `_rotate_right` и `_rotate_left`. Эти операции перенаправляют несколько указателей и обновляют веса и высоты затронутых узлов, выполняясь за $O(1)$. Метод `_balance_node` анализирует баланс-фактор узла и при необходимости

применяет одиночный или двойной поворот, чтобы восстановить баланс AVL-дерева. Все проверки и повороты также выполняются за $O(1)$.

Операция разделения веревки реализована в методе `_split_tree`. Она рекурсивно спускается к листу, содержащему нужную позицию разделения, и затем "разрывает" дерево на две части. В процессе могут создаваться новые узлы и выполняться балансировки. Глубина рекурсии ограничена высотой дерева $O(\log(n))$, а каждая обработка узла требует $O(1)$ времени (за исключением случая разделения листа, где выполняется копирование подстроки - но длина листа ограничена константой `LEAF_SIZE`). Поэтому общая сложность `split_at` (которая является публичным интерфейсом для `_split_tree`) равна $O(\log n)$.

Конкатенация двух веревок в методе `concatenate` выполняется созданием нового корневого узла, который становится родителем для корней двух исходных веревок. После этого вызывается балансировка. В худшем случае может потребоваться несколько поворотов вдоль пути к корню, что в теории могло бы занять $O(\log(n))$, но благодаря свойствам AVL-дерева амортизированная сложность этой операции составляет $O(1)$.

На основе операций разделения и конкатенации строятся более сложные методы: `insert_at` и `delete_range`. Вставка строки по индексу выполняется так: сначала исходная веревка разделяется в точке вставки, затем из вставляемой строки создаётся новая веревка, после чего выполняется конкатенация левой части, новой веревки и правой части. Таким образом, сложность вставки складывается из $O(\log(n))$ для разделения, $O(m)$ для построения веревки из вставляемой строки (где m - её длина) и $O(\log(n))$ для конкатенаций. Итоговая сложность - $O(\log(n+m))$. Удаление диапазона символов требует двух разделений (чтобы выделить удаляемый фрагмент) и одной конкатенации (чтобы соединить оставшиеся части), что даёт сложность $O(\log(n))$.

Вспомогательный метод `print_structure` (и используемая внутри него `traverse_subtree`) выполняет полный обход дерева для сбора информации

о листьях и весах внутренних узлов. Поскольку он посещает каждый узел ровно один раз, его сложность составляет $O(n)$.

Практическое исследование направлено на сравнение времени выполнения в зависимости от размера входных данных (10/1000/100000)

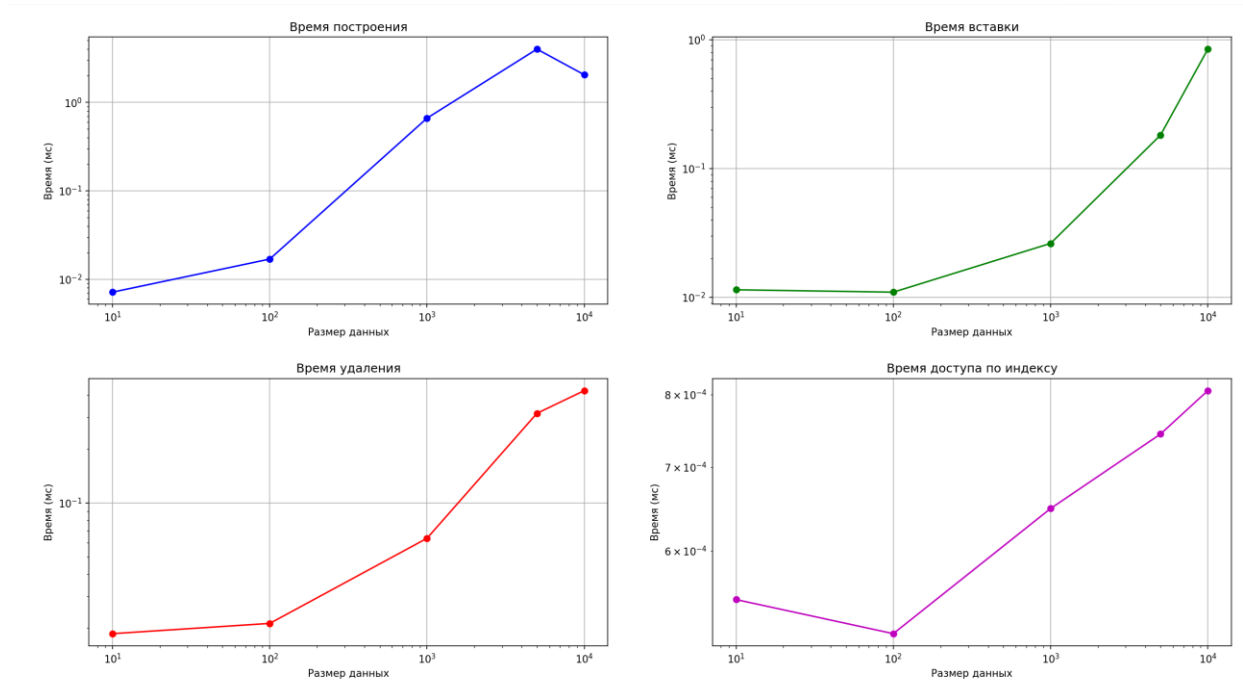


Рисунок 1 – Графики сравнения выполнения времени операций

Вывод.

В рамках выполнения лабораторной работы была успешно разработана и реализована структура данных **Rope**. В процессе работы создан комплекс методов, обеспечивающих полноценное функционирование структуры, включая операции объединения, разделения, добавления и удаления, а также извлечения символов по заданной позиции.

Проведенное исследование включало сравнительный анализ практических характеристик производительности реализованных алгоритмов с теоретическими показателями. Экспериментальные данные продемонстрировали полное соответствие практических результатов теоретическим прогнозам. Все основные операции имеют логарифмическую

временную сложность, что подтвердило эффективность выбранных алгоритмических решений и корректность их программной реализации.

Приложение А

main.py

```
LEAF_SIZE = 8

class Node:
    def __init__(self, string_data=None):
        self.left_child = None
        self.right_child = None
        self.string_data = string_data
        if string_data is not None:
            self.weight = len(string_data)
        else:
            self.weight = 0
        self.height = 1

    def is_leaf(self):
        return self.string_data is not None

    def update_height(self):
        left_h = self.left_child.height
        right_h = self.right_child.height
        self.height = max(left_h, right_h) + 1

    def balance_factor(self):
        left_h = self.left_child.height
        right_h = self.right_child.height
        return left_h - right_h

class Rope:
    def __init__(self, initial_string=""):
        self.root_node = self._build_tree(initial_string)

    def _build_tree(self, s):
        if len(s) <= LEAF_SIZE:
            return Node(s)

        mid_index = len(s) // 2
```

```

        internal_node = Node()
        internal_node.left_child =
self._build_tree(s[:mid_index])
        internal_node.right_child =
self._build_tree(s[mid_index:])
        internal_node.weight =
self._compute_length(internal_node.left_child)
        internal_node.update_height()
        return self._balance_node(internal_node)

def get_char_at(self, index):
    current = self.root_node
    while current and not current.is_leaf():
        if index < current.weight:
            current = current.left_child
        else:
            index -= current.weight
            current = current.right_child

    if not current or not current.string_data or index >=
len(current.string_data):
        raise IndexError("Индекс вне диапазона")

    return current.string_data[index]

def _compute_length(self, node):
    if node.is_leaf():
        return len(node.string_data)
    return self._compute_length(node.left_child) +
self._compute_length(node.right_child)

def _rotate_right(self, y_node):
    x_node = y_node.left_child
    t2_subtree = x_node.right_child

    x_node.right_child = y_node
    y_node.left_child = t2_subtree

    y_node.weight = self._compute_length(y_node.left_child)

```

```

        x_node.weight = self._compute_length(x_node.left_child)

        y_node.update_height()
        x_node.update_height()

        return x_node

def _rotate_left(self, x_node):
    y_node = x_node.right_child
    t2_subtree = y_node.left_child

    y_node.left_child = x_node
    x_node.right_child = t2_subtree

    x_node.weight = self._compute_length(x_node.left_child)
    y_node.weight = self._compute_length(y_node.left_child)

    x_node.update_height()
    y_node.update_height()

    return y_node

def _balance_node(self, node):
    balance = node.balance_factor()

    if balance > 1 and node.left_child.balance_factor() >= 0:
        return self._rotate_right(node)

    if balance < -1 and node.right_child.balance_factor() <=
0:
        return self._rotate_left(node)

    if balance > 1 and node.left_child.balance_factor() < 0:
        node.left_child = self._rotate_left(node.left_child)
        return self._rotate_right(node)

    if balance < -1 and node.right_child.balance_factor() >
0:

```

```

        node.right_child =
self._rotate_right(node.right_child)
        return self._rotate_left(node)

    return node

def _split_tree(self, node, index):
    if node.is_leaf():
        if index <= 0:
            return None, node
        if index >= len(node.string_data):
            return node, None
        left_part = Node(node.string_data[:index])
        right_part = Node(node.string_data[index:])
        return left_part, right_part

    if index <= node.weight:
        left_sub, right_sub =
self._split_tree(node.left_child, index)
        if right_sub is None:
            return left_sub, node.right_child

        new_right = Node()
        new_right.left_child = right_sub
        new_right.right_child = node.right_child
        new_right.weight = self._compute_length(right_sub)
        new_right.update_height()
        return left_sub, self._balance_node(new_right)

    else:
        left_sub, right_sub =
self._split_tree(node.right_child, index - node.weight)
        new_left = Node()
        new_left.left_child = node.left_child
        new_left.right_child = left_sub
        new_left.weight =
self._compute_length(node.left_child)
        new_left.update_height()
        return self._balance_node(new_left), right_sub

```

```

def split_at(self, index):
    if not self.root_node:
        return Rope(), Rope()
    left_root, right_root = self._split_tree(self.root_node,
index)

    left_rope = Rope()
    right_rope = Rope()
    left_rope.root_node = left_root
    right_rope.root_node = right_root
    return left_rope, right_rope

def concatenate(self, other_rope):
    if not self.root_node:
        self.root_node = other_rope.root_node
        return
    if not other_rope.root_node:
        return
    combined_root = Node()
    combined_root.left_child = self.root_node
    combined_root.right_child = other_rope.root_node
    combined_root.weight =
self._compute_length(self.root_node)
    combined_root.update_height()
    self.root_node = self._balance_node(combined_root)

def insert_at(self, index, string_to_insert):
    left_part, right_part = self.split_at(index)
    middle_rope = Rope(string_to_insert)
    left_part.concatenate(middle_rope)
    left_part.concatenate(right_part)
    self.root_node = left_part.root_node

def delete_range(self, start_index, length_to_delete):
    left_part, temp_rope = self.split_at(start_index)
    _, right_part = temp_rope.split_at(length_to_delete)
    left_part.concatenate(right_part)
    self.root_node = left_part.root_node

```



```

def print_structure(self):
    leaf_strings = []
    internal_weights = []

    def traverse_subtree(node):
        if node.is_leaf():
            leaf_strings.append(node.string_data)
            return node.string_data
        internal_weights.append(node.weight)
        return traverse_subtree(node.left_child) +
traverse_subtree(node.right_child)

    full_content = traverse_subtree(self.root_node)
    weight_array = array('i', internal_weights)

    for i, s in enumerate(leaf_strings, 1):
        print(f"Leaf {i}: {s}")
    for i, w in enumerate(weight_array, 1):
        print(f"Node {i}: {w}")
    print(full_content)

def main():
    initial_str = input()
    insert_input = input()
    delete_input = input()

    insert_position = int(insert_input.split()[0])
    insert_text = insert_input.split(' ', 1)[1]

    delete_start, delete_count = map(int, delete_input.split())

    rope_instance = Rope(initial_str)
    rope_instance.print_structure()
    rope_instance.insert_at(insert_position, insert_text)
    rope_instance.print_structure()
    rope_instance.delete_range(delete_start, delete_count)
    rope_instance.print_structure()

```

```

    if __name__ == "__main__":
        main()

```

graphs.py

```

import time
import random
import matplotlib.pyplot as plt

LEAF_SIZE = 8

class Node:
    def __init__(self, string_data=None):
        self.left_child = None
        self.right_child = None
        self.string_data = string_data
        if string_data is not None:
            self.weight = len(string_data)
        else:
            self.weight = 0
        self.height = 1

    def is_leaf(self):
        return self.string_data is not None

    def update_height(self):
        left_h = self.left_child.height if self.left_child else 0
        right_h = self.right_child.height if self.right_child else 0
        self.height = max(left_h, right_h) + 1

    def balance_factor(self):
        left_h = self.left_child.height if self.left_child else 0
        right_h = self.right_child.height if self.right_child else 0
        return left_h - right_h

class Rope:
    def __init__(self, initial_string=""):
        self.root_node = self._build_tree(initial_string)

    def _build_tree(self, s):
        if len(s) <= LEAF_SIZE:
            return Node(s)
        mid_index = len(s) // 2
        internal_node = Node()
        internal_node.left_child = self._build_tree(s[:mid_index])
        internal_node.right_child = self._build_tree(s[mid_index:])
        internal_node.weight =
self._compute_length(internal_node.left_child)
        internal_node.update_height()
        return self._balance_node(internal_node)

    def get_char_at(self, index):
        current = self.root_node
        while current and not current.is_leaf():
            if index < current.weight:

```

```

        current = current.left_child
    else:
        index -= current.weight
        current = current.right_child
    if not current or not current.string_data or index >=
len(current.string_data):
        raise IndexError("Index error")
    return current.string_data[index]

def _compute_length(self, node):
    if node.is_leaf():
        return len(node.string_data)
    return self._compute_length(node.left_child) +
self._compute_length(node.right_child)

def _rotate_right(self, y_node):
    x_node = y_node.left_child
    t2_subtree = x_node.right_child
    x_node.right_child = y_node
    y_node.left_child = t2_subtree
    y_node.weight = self._compute_length(y_node.left_child)
    x_node.weight = self._compute_length(x_node.left_child)
    y_node.update_height()
    x_node.update_height()
    return x_node

def _rotate_left(self, x_node):
    y_node = x_node.right_child
    t2_subtree = y_node.left_child
    y_node.left_child = x_node
    x_node.right_child = t2_subtree
    x_node.weight = self._compute_length(x_node.left_child)
    y_node.weight = self._compute_length(y_node.left_child)
    x_node.update_height()
    y_node.update_height()
    return y_node

def _balance_node(self, node):
    if node.is_leaf():
        return node
    balance = node.balance_factor()
    if balance > 1 and node.left_child.balance_factor() >= 0:
        return self._rotate_right(node)
    if balance < -1 and node.right_child.balance_factor() <= 0:
        return self._rotate_left(node)
    if balance > 1 and node.left_child.balance_factor() < 0:
        node.left_child = self._rotate_left(node.left_child)
        return self._rotate_right(node)
    if balance < -1 and node.right_child.balance_factor() > 0:
        node.right_child = self._rotate_right(node.right_child)
        return self._rotate_left(node)
    return node

def _split_tree(self, node, index):
    if node.is_leaf():
        if index <= 0:
            return None, node
        if index >= len(node.string_data):

```

```

        return node, None
    left_part = Node(node.string_data[:index])
    right_part = Node(node.string_data[index:])
    return left_part, right_part
if index <= node.weight:
    left_sub, right_sub = self._split_tree(node.left_child,
index)

    if right_sub is None:
        return left_sub, node.right_child
    new_right = Node()
    new_right.left_child = right_sub
    new_right.right_child = node.right_child
    new_right.weight = self._compute_length(right_sub)
    new_right.update_height()
    return left_sub, self._balance_node(new_right)
else:
    left_sub, right_sub = self._split_tree(node.right_child,
index - node.weight)
    if left_sub is None:
        return node.left_child, right_sub
    new_left = Node()
    new_left.left_child = node.left_child
    new_left.right_child = left_sub
    new_left.weight = self._compute_length(node.left_child)
    new_left.update_height()
    return self._balance_node(new_left), right_sub

def split_at(self, index):
    if not self.root_node:
        return Rope(), Rope()
    left_root, right_root = self._split_tree(self.root_node,
index)
    left_rope = Rope()
    right_rope = Rope()
    left_rope.root_node = left_root
    right_rope.root_node = right_root
    return left_rope, right_rope

def concatenate(self, other_rope):
    if not self.root_node:
        self.root_node = other_rope.root_node
        return
    if not other_rope.root_node:
        return
    combined_root = Node()
    combined_root.left_child = self.root_node
    combined_root.right_child = other_rope.root_node
    combined_root.weight = self._compute_length(self.root_node)
    combined_root.update_height()
    self.root_node = self._balance_node(combined_root)

def insert_at(self, index, string_to_insert):
    left_part, right_part = self.split_at(index)
    middle_rope = Rope(string_to_insert)
    left_part.concatenate(middle_rope)
    left_part.concatenate(right_part)
    self.root_node = left_part.root_node

```

```

def delete_range(self, start_index, length_to_delete):
    left_part, temp_rope = self.split_at(start_index)
    _, right_part = temp_rope.split_at(length_to_delete)
    left_part.concatenate(right_part)
    self.root_node = left_part.root_node

def length(self):
    return self._compute_length(self.root_node)

def benchmark_build_rope():
    sizes = [10, 100, 1000, 5000, 10000]
    times = []
    for size in sizes:
        test_string = 'a' * size
        start_time = time.time()
        rope = Rope(test_string)
        end_time = time.time()
        times.append((end_time - start_time) * 1000)
    return sizes, times

def benchmark_insert():
    sizes = [10, 100, 1000, 5000, 10000]
    times = []
    for size in sizes:
        rope = Rope('a' * size)
        insert_string = 'b' * 10
        start_time = time.time()
        rope.insert_at(size // 2, insert_string)
        end_time = time.time()
        times.append((end_time - start_time) * 1000)
    return sizes, times

def benchmark_delete():
    sizes = [10, 100, 1000, 5000, 10000]
    times = []
    for size in sizes:
        rope = Rope('a' * size)
        start_time = time.time()
        delete_len = min(10, size // 4)
        rope.delete_range(size // 4, delete_len)
        end_time = time.time()
        times.append((end_time - start_time) * 1000)
    return sizes, times

def benchmark_index():
    sizes = [10, 100, 1000, 5000, 10000]
    times = []
    for size in sizes:
        rope = Rope('a' * size)
        start_time = time.time()
        for _ in range(50):
            index = random.randint(0, size - 1)
            rope.get_char_at(index)
        end_time = time.time()
        times.append(((end_time - start_time) / 50) * 1000)
    return sizes, times

def plot_results():

```

```

build_sizes, build_times = benchmark_build_rope()
insert_sizes, insert_times = benchmark_insert()
delete_sizes, delete_times = benchmark_delete()
index_sizes, index_times = benchmark_index()

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12,
8))

ax1.plot(build_sizes, build_times, 'bo-')
ax1.set_xscale('log')
ax1.set_yscale('log')
ax1.set_title('Время построения')
ax1.set_xlabel('Размер данных')
ax1.set_ylabel('Время (мс)')
ax1.grid(True)

ax2.plot(insert_sizes, insert_times, 'go-')
ax2.set_xscale('log')
ax2.set_yscale('log')
ax2.set_title('Время вставки')
ax2.set_xlabel('Размер данных')
ax2.set_ylabel('Время (мс)')
ax2.grid(True)

ax3.plot(delete_sizes, delete_times, 'ro-')
ax3.set_xscale('log')
ax3.set_yscale('log')
ax3.set_title('Время удаления')
ax3.set_xlabel('Размер данных')
ax3.set_ylabel('Время (мс)')
ax3.grid(True)

ax4.plot(index_sizes, index_times, 'mo-')
ax4.set_xscale('log')
ax4.set_yscale('log')
ax4.set_title('Время доступа по индексу')
ax4.set_xlabel('Размер данных')
ax4.set_ylabel('Время (мс)')
ax4.grid(True)

plt.tight_layout()
plt.savefig('rope_performance.png')
plt.show()

if __name__ == "__main__":
    plot_results()

```

Приложение Б

test.py

```
import pytest
from main import Rope, Node, LEAF_SIZE

class TestNode:
    def test_node_leaf_initialization(self):
        node = Node("hello")
        assert node.string_data == "hello"
        assert node.weight == 5
        assert node.height == 1
        assert node.is_leaf()
        assert node.left_child is None
        assert node.right_child is None

    def test_node_internal_initialization(self):
        node = Node()
        assert node.string_data is None
        assert node.weight == 0
        assert node.height == 1
        assert not node.is_leaf()
        assert node.left_child is None
        assert node.right_child is None

class TestRopeBasic:
    def test_single_char_rope(self):
        rope = Rope("a")
        assert rope.root_node.string_data == "a"
        assert rope.root_node.weight == 1
        assert rope.root_node.is_leaf()

    def test_rope_within_leaf_size(self):
        rope = Rope("test123")
        assert rope.root_node.string_data == "test123"
        assert rope.root_node.weight == 7

    def test_rope_exceeds_leaf_size(self):
        rope = Rope("a" * (LEAF_SIZE + 5))
        assert rope.root_node.weight > 0
        assert not rope.root_node.is_leaf()

    def test_rope_build_large_string(self):
        rope = Rope("abcdefghijklmnopqrstuvwxyz")
        assert rope.root_node is not None

class TestRopeGetCharAt:
    def test_get_char_from_short_string(self):
        rope = Rope("hello")
        assert rope.get_char_at(0) == 'h'
        assert rope.get_char_at(1) == 'e'
        assert rope.get_char_at(4) == 'o'

    def test_get_char_from_alphabet(self):
```

```

    rope = Rope("abcdefghijklmnopqrstuvwxyz")
    assert rope.get_char_at(0) == 'a'
    assert rope.get_char_at(25) == 'z'
    assert rope.get_char_at(12) == 'm'

def test_get_char_index_out_of_bounds(self):
    rope = Rope("test")
    with pytest.raises(IndexError):
        rope.get_char_at(10)

def test_get_char_from_empty_rope(self):
    rope = Rope("")
    with pytest.raises(IndexError):
        rope.get_char_at(0)

class TestRopeSplitAt:
    def test_split_at_zero(self):
        rope = Rope("hello world")
        left, right = rope.split_at(0)
        assert right.get_char_at(0) == 'h'
        assert right.get_char_at(10) == 'd'

    def test_split_at_end(self):
        rope = Rope("hello world")
        left, right = rope.split_at(11)
        assert left.get_char_at(0) == 'h'
        assert left.get_char_at(10) == 'd'

    def test_split_in_middle(self):
        rope = Rope("hello world")
        left, right = rope.split_at(5)
        assert left.get_char_at(0) == 'h'
        assert left.get_char_at(4) == 'o'
        assert right.get_char_at(0) == ' '
        assert right.get_char_at(5) == 'd'

    def test_split_small_string(self):
        rope = Rope("abc")
        left, right = rope.split_at(1)
        assert left.get_char_at(0) == 'a'
        assert right.get_char_at(0) == 'b'
        assert right.get_char_at(1) == 'c'

class TestRopeConcatenate:
    def test_concatenate_two_strings(self):
        rope1 = Rope("hello")
        rope2 = Rope(" world")
        rope1.concatenate(rope2)
        for i, char in enumerate("hello world"):
            assert rope1.get_char_at(i) == char

    def test_concatenate_with_empty(self):
        rope1 = Rope("hello")
        rope2 = Rope("")
        rope1.concatenate(rope2)
        assert rope1.get_char_at(0) == 'h'

```



```

        assert rope1.get_char_at(4) == 'o'

    def test_concatenate_empty_with_string(self):
        rope1 = Rope("")
        rope2 = Rope("world")
        rope1.concatenate(rope2)
        assert rope1.get_char_at(0) == 'w'
        assert rope1.get_char_at(4) == 'd'

class TestRopeInsertAt:
    def test_insert_at_beginning(self):
        rope = Rope("world")
        rope.insert_at(0, "hello ")
        for i, char in enumerate("hello world"):
            assert rope.get_char_at(i) == char

    def test_insert_at_end(self):
        rope = Rope("hello")
        rope.insert_at(5, " world")
        for i, char in enumerate("hello world"):
            assert rope.get_char_at(i) == char

    def test_insert_in_middle(self):
        rope = Rope("heworld")
        rope.insert_at(2, "llo ")
        for i, char in enumerate("hello world"):
            assert rope.get_char_at(i) == char

class TestRopeDeleteRange:
    def test_delete_from_start(self):
        rope = Rope("hello world")
        rope.delete_range(0, 6)
        for i, char in enumerate("world"):
            assert rope.get_char_at(i) == char

    def test_delete_from_end(self):
        rope = Rope("hello world")
        rope.delete_range(6, 5)
        for i, char in enumerate("hello "):
            assert rope.get_char_at(i) == char

class TestRopeComplexOperations:
    def test_multiple_operations_simple(self):
        rope = Rope("start")
        rope.insert_at(0, "begin ")
        rope.insert_at(rope._compute_length(rope.root_node), " end")
        result = ''.join(rope.get_char_at(i) for i in
range(rope._compute_length(rope.root_node)))
        assert result == "begin start end"

class TestRopeEdgeCases:
    def test_very_long_string(self):
        text = "x" * 1000
        rope = Rope(text)

```

```

    assert rope.get_char_at(0) == 'x'
    assert rope.get_char_at(999) == 'x'
    assert rope.get_char_at(500) == 'x'

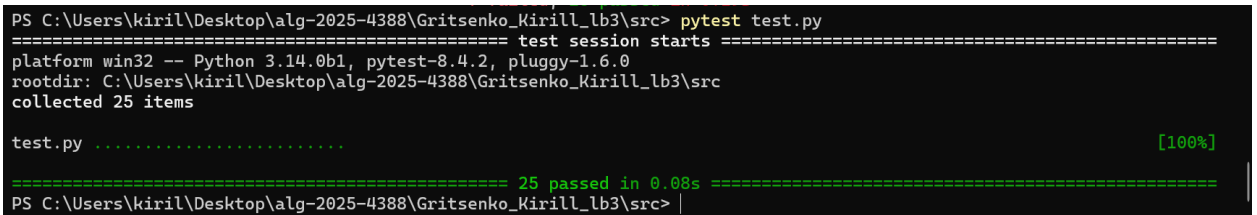
    def test_special_characters_in_string(self):
        text = "Hello\tWorld\nNew\rLine"
        rope = Rope(text)
        for i in range(len(text)):
            assert rope.get_char_at(i) == text[i]

def get_rope_content(rope):
    if not rope.root_node:
        return ""
    length = rope._compute_length(rope.root_node)
    return ''.join(rope.get_char_at(i) for i in range(length))

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

Результат:



```

PS C:\Users\kiril\Desktop\alg-2025-4388\Gritsenko_Kirill_lb3\src> pytest test.py
===== test session starts =====
platform win32 -- Python 3.14.0b1, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\kiril\Desktop\alg-2025-4388\Gritsenko_Kirill_lb3\src
collected 25 items

test.py ..... [100%]

===== 25 passed in 0.08s =====
PS C:\Users\kiril\Desktop\alg-2025-4388\Gritsenko_Kirill_lb3\src>

```

Рис.1 – Проверка работоспособности